

EECS 3216 Lab 4 – FPGA-Based Parking Meter System Design

Instructor: Alireza Ahrar

1 Objective

The purpose of this lab is to design and implement a customized parking meter system based on the provided specifications onto the DE10-Lite board [1], utilizing either SystemVerilog, Verilog, or VHDL. After completing the lab, students must demonstrate the correct functionality of the program and submit a report, along with the corresponding source code, on the eClass platform. The primary source code file should be named `student#1_student#2_Lab4.v`, and any additional files generated during the lab should be compressed into a zip file and submitted.

2 Introduction

When using a parking meter, adding a coin (`KEY1`) to an available space increases the parking time by 10 seconds. In this lab, three spaces are available and controlled by slide switches (`SW2`, `SW1`, `SW0`). Moving the switches up occupies a space, turns on the corresponding LED, and decreases the value displayed on the 7-segments (`HEX3`) for available spaces.

To deposit a coin, you must first select the space using slide switches (`SW9`, `SW8`), and the 7-segment (`HEX2`) should display the chosen space. Coin deposition is simulated by pressing the `KEY1` push-button. Each press signifies a coin deposit, adding 10 seconds to the timer, which starts counting down 1 second after the first deposit. The remaining time continuously displays on two 7-segment displays (`HEX1`, `HEX0`) and reaches a maximum of 99 seconds. Any deposit exceeding this limit saturates at 99 seconds.

When the remaining time drops below 10 seconds, the corresponding LED blinks with a 0.5-second interval (0.5 seconds on and 0.5 seconds off) until it reaches 0. After that, if no coin is inserted, the LED blinks with a 0.1-second interval.

3 Specification

Design and implement a circuit in HDL to achieve the following functionality and patterns:

1. Generate an asynchronous `RESET` signal using push-button `KEY0` to reset the system and display the pattern in Fig. 1.
2. Three slide switches (`SW2`, `SW1`, `SW0`), and LEDs (`LEDR2`, `LEDR1`, `LEDR0`) are responsible for showing the available spaces on 7-segments (`HEX5`, `HEX4`, `HEX3`)—Show `OP` for open and `FL` for full.
3. Two switches (`SW9`, `SW8`) are responsible for selecting the current space to increase the time.

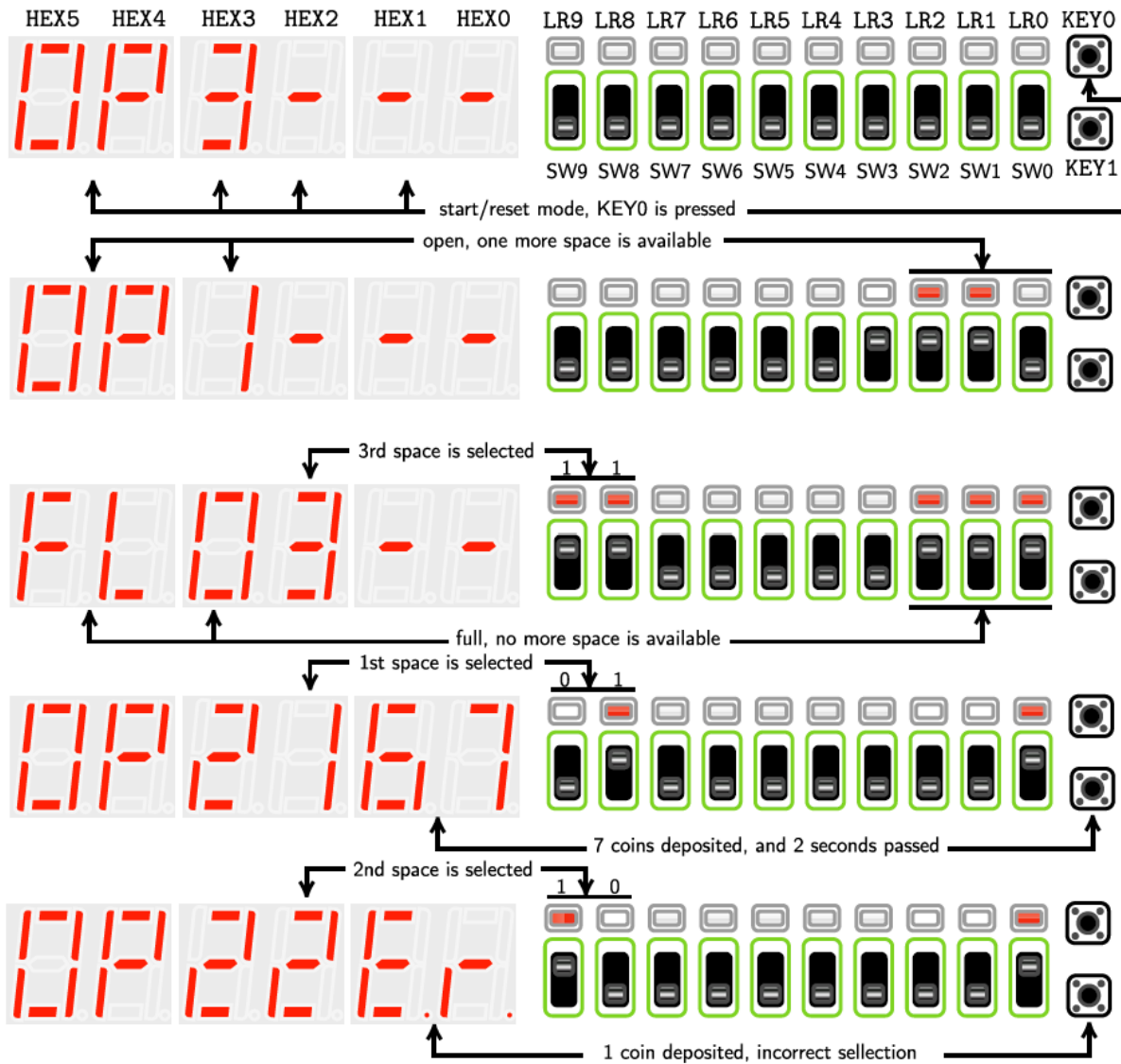


Figure 1: Parking system pattern.

- KEY1 increases the time by 10 seconds, and the timer (HEX1, HEX0) starts 1 second after the first deposit. Additional deposits can be made during the countdown process.
- The time decreases every second, and the updated timer should be displayed on (HEX1, HEX0).
- The time is displayed as decimal, and the maximum allowable time is 99 seconds.
- When the remaining time reaches 10, the corresponding LED blinks with a 0.5-second interval.
- When the remaining time reaches 00, the countdown stops, and the corresponding LED blinks with a 0.1-second interval if no coin is inserted after that point.
- When an unoccupied but selected space is detected, the program should make the corresponding LED blink three times, and an error message should be displayed on the timer segments.

10. For any unspecified behavior, you may make your own assumptions and make sure to address them in the report as well.

The design could be in pseudo-code, FSM, or whatever you choose; the only condition is that if the TA reads the design, they can follow and understand your code.

4 Submission Guidelines

1. Students must submit a report following the format below in the eClass submission box:
 - **Introduction:** What is the lab about? Discuss the general idea and purpose of the lab.
 - **Explanation:** Discuss the main idea and how your algorithm addresses the specification. Specify the resources you utilized, etc.
 - **Conclusion:** Discuss your experience in the lab, what you learned, any difficulties encountered, etc.
2. The main source code file should be named `student#1_student#2_Lab4.v`.
3. Any additional files created during the lab should also be included in the submission as a zip archive.

5 Marking Scheme

The labs will be graded based on the successful demo (shown to the TAs during the lab) and quality of the report along with the correctness of the submitted codes. Each lab will have 4% of total mark. Your demo to TA has 2% and the other 2% is for the lab report and code.

The lab implementation is marked out of 4 points:

- Code functionality (4 points)
 1. 4 points if the code works exactly as expected.
 2. 3 points if all of the items work but there are some bugs.
 3. 2 points if three items work.
 4. 1 points if two items work.
 5. 0.5 point if no items work but you can program the FPGA.
 6. 0 points if you cannot even program the FPGA.

Credits

This series of labs was originally developed by Mokhtar Aboelaze at York University. In Winter 2024, Amirali Amirsoleimani instructed the EECS 3216 course, and it was revised during that term by Hamid Rahimian Kalatehbali, who served as the teaching assistant. In Fall 2026, this version was further updated and developed by Alireza Ahrar, with contributions from teaching assistants Danial Baharlouei and Mohammadreza Hajipour.

References

- [1] DE10-Lite, *Terasic Max 10 10M50DAF484C7G Device*, 2022.